

46. Un objet qui tombe

La pesanteur, en seizièmes de pixel.

1. Ce qu'il faut comprendre

Un objet qui tombe ne descend pas à vitesse constante : il **accélère**. Toute la physique tient en deux additions — la pesanteur s'ajoute à la vitesse, la vitesse s'ajoute à la position — et c'est de les faire **dans cet ordre**, à chaque image, qui fabrique la courbe d'une chute.

Reste un ennui : la console **ne connaît pas la virgule**. Au premier instant, la balle avance de deux seizièmes de pixel — arrondi, cela fait zéro, et elle ne partirait jamais.

On garde donc la position en **deux morceaux** : les pixels dans `y`, et les seizièmes de pixel dans `frac`. Seul `y` est montré ; `frac` est la mémoire de ce qui n'est pas encore un pixel entier, et qui le deviendra.

`frac >> 4` est une division par seize, `frac & 15` en est le reste. Le processeur ne sait pas diviser ; décaler de quatre bits, en revanche, lui coûte un seul tour d'horloge.

La chute dure **trois quarts de seconde**. Une démonstration qui ne se joue qu'une fois ne se voit pas : on compte donc les images passées au sol, et **au bout de quarante la balle est renvoyée en haut**. Elle tombe en boucle, et l'on peut la regarder tomber autant qu'on veut.

La relance remet **les trois** : la hauteur, les seizièmes et la vitesse. Le sol vient de les mettre à zéro une ligne plus haut — mais un départ qui compte sur le ménage fait par quelqu'un d'autre n'est pas un départ, et le jour où l'on relancera d'ailleurs la balle partira du ciel avec la vitesse qu'elle avait en s'écrasant.

`G` vaut 2, et ce nombre est le seul réglage : à 1 la balle flotte comme sur la Lune, à 6 elle tombe comme une enclume. C'est là, et nulle part ailleurs, que se décide ce qu'on ressent.

Et 9,81 dans tout ça ? Elle est dans `G`. Le processeur n'a ni virgule ni mètres : `G` se compte en seizièmes de pixel par image, par image, soit 450 pixels par seconde carrée à soixante images par seconde. Décidez qu'un mètre fait 46 pixels, et ces 450 pixels deviennent **9,81 m/s²** — l'écran mesure alors 3,10 m de haut, et la chute de 2,60 m dure les trois quarts de seconde que la vraie physique annonce. La pesanteur est la bonne ; c'est l'unité qui a changé.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **721 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      57 o ← la console, à l'allumage – la mise en route de la console
main()         159 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
RangerLutins   11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles   94 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         240 o ← CopierTuiles – 14 lettres sur 43, et 1 tuile dessinée
DonneesTransfert 10 o ← InstallerTransfert – la petite routine qui copie les lutins
"CHUTE LIBRE"  11 o ← texte() – un texte gravé
"ET CA RECOMMENCE" 16 o ← texte() – un texte gravé
"#####..."  20 o ← texte() – un texte gravé
```

Lettres gravées : ABCEHILMNORTU#

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes

Fonctions du langage appelées :

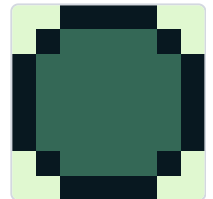
```
texte() × 3    33 o sur place, dans main() + EcrireTexte (10 o)
sprite() × 1   23 o sur place, dans main()
image() × 1     6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>     // écrit un texte à l'écran
#include <sprite>    // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — BALLE

```
Tuile BALLE = {
  "00333300",
  "03222230",
  "32222223",
  "32222223",
  "32222223",
  "32222223",
  "32222223",
  "03222230",
  "00333300",
};
```



à l'écran

3. UNE CONSTANTE — CIEL

```
const uint8_t CIEL = 8;    // le haut de la balle quand on la lâche
```

4. UNE CONSTANTE — SOL

```
const uint8_t SOL = 128;   // le haut de la balle quand elle touche terre
```

5. UNE CONSTANTE — G

```
const uint8_t G = 2;       // la pesanteur, en seizièmes de pixel par image
```

6. UNE CONSTANTE — ATTENTE

```
const uint8_t ATTENTE = 40; // les images passées au sol avant de recommencer
```

7. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t y = CIEL; // les pixels – ce que l'écran montre
    uint8_t frac = 0; // les seizièmes – ce qu'il ne sait pas montrer
    uint8_t vy = 0;    // la vitesse, en seizièmes de pixel par image
    uint8_t pose = 0;  // depuis combien d'images elle est posée

    texte(4, 2, "CHUTE LIBRE");
    texte(2, 4, "ET CA RECOMMENCE");
    texte(0, 17, "#####");

    while (true) {
        image();

        vy = vy + G;

        frac = frac + vy;
        y = y + (frac >> 4);
        frac = frac & 15;

        if (y >= SOL) {
            y = SOL;
            frac = 0;
            vy = 0;
            pose = pose + 1;
        }

        if (pose > ATTENTE) {
            y = CIEL;
            frac = 0;
            vy = 0;
            pose = 0;
        }

        sprite(0, 76, y, BALLE);
    }

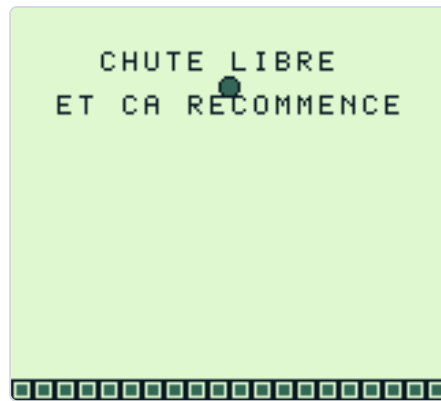
    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



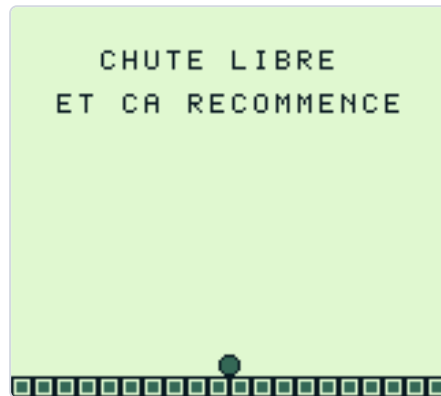
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

La balle part lentement, prend de la vitesse, s'arrête net sur le sol — puis remonte d'un coup et recommence.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ elle part du haut de l'écran
- ✓ elle descend jusqu'au sol
- ✓ elle accélère en tombant (4 pixels, puis 13)
- ✓ elle marque un temps d'arrêt (86 images au sol)
- ✓ et elle recommence toute seule (2 chutes en 200 images)

Ensuite : **47. Un personnage de seize** — chapitre 7 : Le mouvement. · gameboy3 — écrire une cartouche Game Boy en C++